LABORATORIO 1

Introducción a la placa de desarrollo y a VSCode + ESP-IDF

Angelo Gauthier

María Pilar Martínez

Ignacio Negri

Introducción

El objetivo de este informe consiste en utilizar y estudiar el hardware de la placa de desarrollo del curso (ESP32-S2-KALUGA-1) y el entorno de desarrollo ESP-IDF en VSCode. Se busca escribir código en lenguaje C para aplicaciones embebidas, desarrollar funciones básicas de E/S, y emplear las herramientas de compilación, flasheo y depuración que ofrece el entorno de desarrollo.

Primera parte: introducción al entorno ESP-IDF + VSCode

<u>Creación de un proyecto:</u>

Al crear un proyecto se despliegan los siguientes directorios en la vista *Explorador* del *workspace* de VSCode. Allí hay distintos archivos y carpetas que sirven para programar, configurar y compilar la aplicación en la placa ESP32-S2.

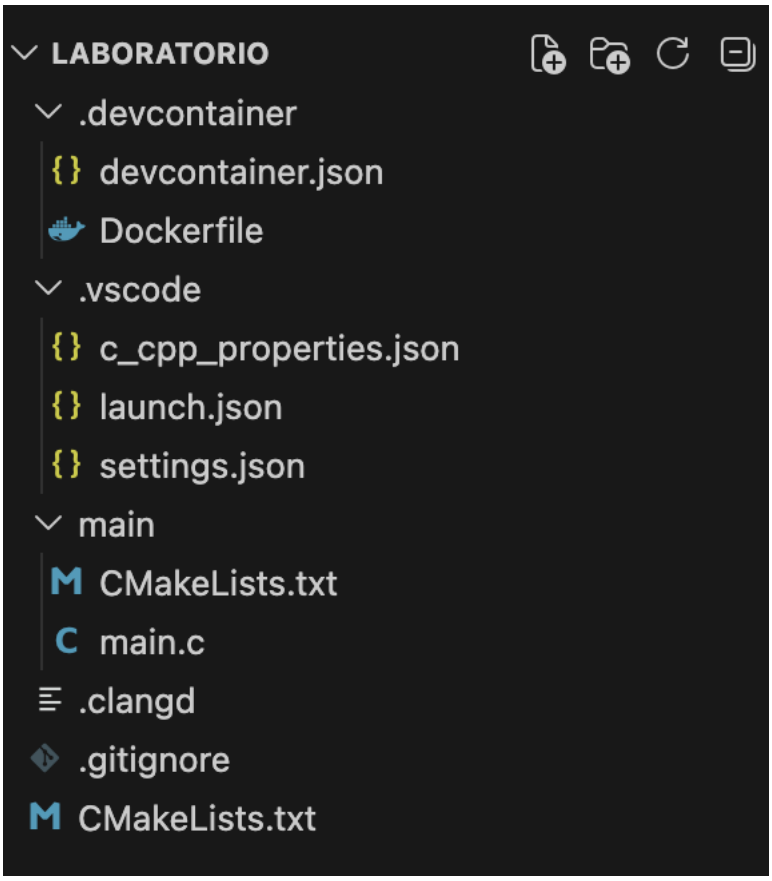

Figura 1: árbol de directorios al crear el proyecto

Debajo en la *Figura 1, se puede observar la carpeta main,* es una de las más importantes ya que contiene el código principal del programa. Dentro se encuentra *main.c,* el cual implementa la función *app_main(),* que describe el código que se ejecutara en la placa. Junto con eso, se tiene el archivo *CMakeLists.txt.* Este le indica al sistema de compilación que archivos debe tomar en cuenta para construir el proyecto.

En la base del proyecto se encuentra otro archivo *CMakeLists.txt*, que, si bien cumple una función similar al anterior, lo hace a nivel general del proyecto completo, Sirve para

organizar la compilación y conectar todas las partes del programa de manera que ESP-IDF sea capaz de compilarlo correctamente. [1]

La carpeta .vscode guarda configuraciones propias del editor Visual Studio Code y contiene los siguientes archivos:

- c_cpp_properties.json
- launch.json
- settings.json

La carpeta .devcontainer tiene archivos relacionados con Docker (contenedores de desarrollo). Esto permite abrir el proyecto en un entorno ya preparado, con herramientas instaladas y configuradas. Dentro de la carpeta aparecen los siguientes archivos:

- devcontainer.json
- dockerfile

Finalmente, aparecen dos archivos: .gitignore y .clangd. El primero se utiliza para git e indica que archivos no se deben subir al repositorio, por ejemplo, archivos temporales o los que son generados automáticamente. El otro archivo está relacionado con ayudas de programación en C/C++, como detección de errores o sugerencias dentro del editor.

Luego de ese análisis, se compilo el proyecto tal cual estaba apretando *build.* Al compilar, aparecieron nuevos archivos y carpetas que fueron generados automáticamente por ESP-IDF. [2]

El cambio principal fue la creación de la carpeta *build*, donde se almacenan todos los archivos necesarios para la compilación y posterior carga del programa en la placa. Dentro de esta carpeta se generan archivos binarios, tablas de particiones, mapas de memoria, archivos temporales y otros elementos que son internos del proceso. [3]

A su vez, también apareció al final de todo el archivo *sdkconfig*. Este guarda la configuración actual del proyecto. En este archivo se almacenan parámetros como el microcontrolador seleccionado, opciones de compilación y más configuraciones del entorno.

Además, el compilador mostro información en la terminal indicando el avance del proceso, los componentes compilados y el resultado final de la compilación. También mostro el tamaño del programa generado y si la compilación fue exitosa. Por ejemplo, una de las líneas que salió en la terminal fue el tamaño del siguiente archivo: "L*aboratorio.bin binary size 0x27760 bytes*".

<u>Parametros de configuracion (sdkconfig):</u>

Como se mencionó anteriormente, el archivo sdkconfig contiene todas las configuraciones del proyecto en ESP-IDF, las cuales se generan y modifican mediante la herramienta menuconfig. En este archivo se definen distintos parámetros del sistema, del microcontrolador y de los periféricos utilizados.

Algunas opciones relevantes que se pueden encontrar son, por ejemplo, la selección del microcontrolador con CONFIG_IDF_TARGET="esp32s2", que indica que el proyecto está configurado para ese chip. También aparecen configuraciones del sistema como la frecuencia del CPU (CONFIG_ESP_DEFAULT_CPU_FREQ_MHZ=160), que sirve para determinar la base de tiempo del sistema y afectan funciones de delay.

Además, se pueden ver opciones relacionadas con el nivel de logs para depuración como CONFIG_LOG_DEFAULT_LEVEL_INFO, y configuraciones de la memoria flash, por ejemplo: CONFIG_ESPTOOLPY_FLASHSIZE="2MB".

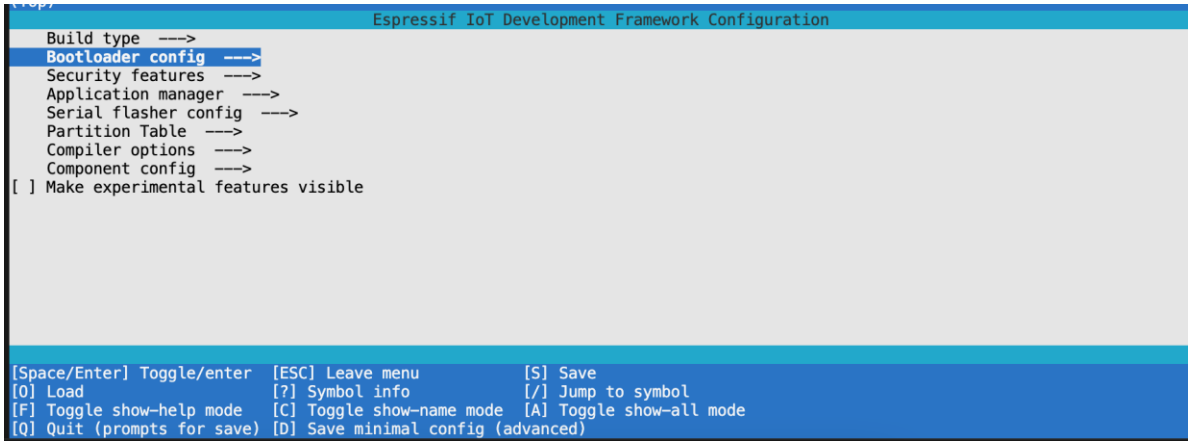

Figura 2: menú de configuración desplegable que aparece en la terminal luego del comando idf.py menuconfig

Por otro lado, al modificar alguna opción mediante el menú de configuración (ver la Figura 2) y guardar los cambios, se actualiza el archivo sdkconfig y se genera automáticamente un archivo *sdkconfig.old*. Este último contiene la configuración anterior, lo que permite comparar cambios.

En el archivo *sdkconfig*, las opciones que aparecen igualadas a "y" indican que están habilitadas, mientras que las opciones deshabilitadas aparecen comentadas con "is not set". [3]

<u>Compilación y análisis de mapa de memoria:</u>

Se modifico el *main.c* agregando lo indicado en la letra. Al compilarlo, se analizaron los archivos *flasher_args.json* y *project.description.json.* Ambos participan del proceso del flasheo, el cual se encarga de transferir los archivos compilados desde la computadora, hacia la memoria flash de la placa para que, en este caso, la ESP32-S2 pueda ejecutar el programa.

El archivo flasher_args.json contiene los argumentos que se necesitan para grabar el programa en la placa. En él se especifica que archivos binarios se deben usar y en que direcciones de memoria deben cargarse. Por ejemplo, allí se indican el archivo de "bootloader", "app" y "partition-table". Su función es facilitar el proceso de flasheo, ya que le dice a la herramienta que debe escribir en la memoria flash del microcontrolador y donde hacerlo. [2]

El archivo project_description.json contiene información general sobre el proyecto compilado. Allí se incluyen datos como "project_name", "project_path" y otros detalles del build. No se usa directamente para grabar la placa, pero sirve como descripción del proyecto generado y ayuda a identificar como fue compilado.

```
 .bss.exampleArray
                0x00000000        0xc esp-idf/main/libmain.a(main.c.obj)
 .bss.exampleData
                0x00000000        0x4 esp-idf/main/libmain.a(main.c.obj)
```

Figura 3: se localizaron en la terminal las variables declaradas en el main.c

Las variables declaradas anteriormente según lo indicado en la letra fueron *exampleData* y *exampleArray.* Estas fueron encontradas dentro del archivo *Laboratorio.map*, ambas en la sección *.bss*. Esta es la zona donde se almacenan las variables globales no inicializadas.

La variable *exampleArray,* al estar declarada como un arreglo de 12 caracteres ocupa 12 bytes de memoria (aparece como 0xc en hexadecimal). Mientras que la variable *exampleData,* de tipo *int,* ocupa 4 bytes (0x4 en hexadecimal).

En la salida observada del archivo *.map* como se ve en la *Figura 3,* ambas aparecen asociadas al archivo objeto *main.c.obj*, lo que confirma que fueron ubicadas correctamente dentro del programa compilado. Sin embargo, no se logró obtener la dirección de memoria exacta, no se pudo determinar con precisión a partir de esa salida.

Al modificar el tamaño de la variable y recompilar se observó que el tamaño de *exampleArray* disminuye de 12 a 10 bytes (pasando de 0xc a 0xa en hexadecimal).

<u>Flashear y monitorear:</u>

Durante el proceso de flasheo, la terminal muestra información relevante sobre la comunicación con la placa y la escritura en la memoria flash. Entre los datos que se observan se encuentran el puerto utilizado (COM3 o COM4 en nuestro caso), la velocidad de comunicación, la conexión con el dispositivo, el tipo de chip detectado y algunas de sus características, el tamaño de la memoria flash y el progreso de la transferencia de datos. También se indican las direcciones de memoria en las que se escribe cada sección y se verifica que la grabación se haya realizado correctamente. [4]

En este proceso se transfieren tres secciones principales. En primer lugar, el bootloader, que es un programa pequeño que se ejecuta al encender el microcontrolador y se encarga de inicializar el sistema y cargar la aplicación principal. Luego se escribe la partition-table, que define cómo está organizada la memoria flash y qué espacio ocupa cada componente. Por último, se transfiere la app, que corresponde al programa desarrollado y que será ejecutado por el microcontrolador. En la terminal se nos muestra información de esa escritura, datos como la cantidad de bits, las direcciones y tiempos de escritura.

```
Serial port COM4:
Connecting....
Connected to ESP32-S2 on COM4:
Chip type:          ESP32-S2 (revision v0.0)
Features:           Wi-Fi, Single Core, 240MHz, No Embedded Flash, No Embedded PSRAM, ADC and temperature sensor calibration in BLK2 of eFuse V1
Crystal frequency:  40MHz
MAC:                7c:df:a1:02:c2:ac
```

Figura 4: Imagen de consola luego de hacer flash.

En la *Figura 4* se observa lo mostrado en la consola, en cuando a los puertos utilizados, información del chip y sus características, frecuencias, etc., como ya se mencionó anteriormente.

```
Writing 'bootloader/bootloader.bin' at 0x00001000...
SHA digest in image updated.
Flash will be erased from 0x00001000 to 0x00006fff...
Compressed 22112 bytes to 14042...
Writing at 0x00001000 [                              ]   0.0% 0/14042 bytes...
Writing at 0x00006660 [==============================] 100.0% 14042/14042 bytes...
Wrote 22112 bytes (14042 compressed) at 0x00001000 in 0.7 seconds (262.6 kbit/s).
Verifying written data...
Hash of data verified.
```

Figura 5: Imagen de la consola luego de hacer flash, específicamente sobre el bootloader.

En la *Figura 5*, se observa como ya se mencionó antes, la escritura del bootloader en las direcciones de memoria especificadas, con información de tiempos y velocidades de escritura. Análogamente, se escriben el partition-table y la app.

Al arrancar la placa, el monitor muestra una serie de mensajes automáticos generados por el sistema. En la salida aparecen mensajes relacionados con la inicialización de la memoria RAM disponible, entre otras cosas más de la memoria flash y el modo de funcionamiento. Por ejemplo, se visualizaron líneas como "*heap_init*", "*spi_flash*", y "*main_task: Calling app_main()*". [5]

Estos mensajes no fueron escritos manualmente en el archivo del main.c, fueron generados automáticamente por el framework durante la inicialización del hardware, la memoria y el sistema operativo embebido. Para agregar, el monitor finalizo con el mensaje "*main_task: Returned form app_main()"* lo que indica que la función termino su ejecución y devolvió el control al sistema.

Posteriormente, se puede ver en la *Figura 6*, como al introducir un "*printf()"* en el *main.c*, compilar nuevamente y flashear, el monitor devuelve ese mensaje. Lo que confirma que todos los mensajes anteriores fueron creados automáticamente por ESP-IDF.

```
I (214) efuse_init: Chip rev:         v0.0
I (218) heap_init: Initializing. RAM available for dynamic allocation:
I (224) heap_init: At 3FFC21F0 len 00039E10 (231 KiB): RAM
I (229) heap_init: At 3FFFC000 len 00003A10 (14 KiB): RAM
I (234) heap_init: At 3FF9E000 len 00001FE8 (7 KiB): RTCRAM
I (241) spi_flash: detected chip: gd
I (243) spi_flash: flash io: dio
I (246) main_task: Started on CPU0
I (246) main_task: Calling app_main()
Mensaje de prueba para visualizar en el monitor
I (256) main_task: Returned from app_main()
```

Figura 6: imagen del monitor una vez cambiado el main.c

Conclusión

En esta primera parte del laboratorio se logró conocer el entorno de desarrollo ESP-IDF y su integración con VSCode, comprendiendo la estructura básica de un proyecto para sistemas embebidos. Se analizaron los archivos y carpetas principales generados automáticamente, asi como la funcionalidad que cumplen durante la compilación y la configuración del sistema.

Asimismo, se estudió el proceso de flasheo del programa hacia la placa y la utilización del monitor como herramienta de depuración y seguimiento de la ejecución. Hubo una aproximación al mapa de memoria, y se analizó como el compilador organiza variables y recursos dentro del programa generado.

En líneas generales, esta primera parte consistió en adquirir una visión completa del proceso de trabajo inicial en desarrollo embebido. Lo que consiste en crear, configurar, compilar, cargar y verificar una aplicación en hardware real.

Segunda Parte - "Hola Mundo" embebido: parpadeo de LED RGB

LED RGB WS2812C

El protocolo de comunicación del WS2812C es serial, de manera unidireccional, sin el uso de un clock y basado en modulación por ancho de pulso. Esto significa que los 24 bits (8 por color) se envían a través de un solo cable, no en paralelo. El led no responde al microcontrolador, escucha. En lugar de utilizar un clock, usa tiempos exactos de pulsos para distinguir entre un 0 y un 1. El led interpreta los bits que le envía el microcontrolador midiendo la duración de los pulsos alto y bajo. [6]

Según el datasheet, los tiempos son del orden de ns o us como se puede observar en la *Tabla 1*:

Data Transfer Time

T0H	0 code, high voltage time	220ns~380ns
T1H	1 code, high voltage time	580ns~1μs
T0L	0 code, low voltage time	580ns~1μs
T1L	1 code, low voltage time	220ns~420ns
RES	Frame unit, low voltage time	>280μs

Tabla 1: tabla extraída del datasheet

Se creo un pseudocódigo para controlar el led desde el microcontrolador:

```
func enviar_bit(bit):
    if bit == 1:
        poner_pin_alto()
        esperar(T1H)
        poner_pin_bajo()
        esperar(T1L)
    else:
        poner_pin_alto()
        esperar(T0H)
        poner_pin_bajo()
        esperar(T0L)
```

```
func enviar_byte(byte):
    for i desde 7 hasta 0:
        enviar_bit( (byte >> i) & 1)

func enviar_color(R, G, B):
    enviar_byte(R)
    enviar_byte(G)
    enviar_byte(B)


func reset_leds():
    poner_pin_bajo()
    esperar(Tiempo>Treset)


func main():
    inicializar_pin_salida()

    reset_leds()

    enviar_color(255, 0, 0)   // Rojo
    enviar_color(0, 255, 0)   // Verde
    enviar_color(0, 0, 255)   // Azul

    reset_leds()
```

Librería de manejo del LED embebido

Luego de realizar esto se implementaron las librerias del manejo del LED embebido inluyendo dos archivos en una carpeta llamada *rgb_led*. Uno de los archivos es *rgb_led.h* en el que se declararon las funciones a utilizar en el archivo *rgb_led.c*. Junto con esto se incluyó un archivo *CMakeLists.txt* el cual contiene lo explicitado en la letra del Laboratorio 1.

Librería de delay

A su vez, se estructuro otra carpeta llamada *delay*, la cual expone tanto la librería *delay.h* y el archivo *delay.c* junto con su *CMakeLists.txt.* Todos estos archivos y librerias creadas se encuentran en una carpeta del proyecto llamada *components.*

Implementación del parpadeo

Para poder trabajar con lo proporcionado por los docentes, se hicieron varios cambios en el driver del LED WS2812 para adaptarlo a la API nueva de RMT en ESP-IDF v6, ya que el código original estaba basado en la implementación vieja. Antes se incluía *driver/rmt.h* y se trabajaba con tipos como rmt_item32_t, además de usar funciones como *rmt_write_sample, rmt_wait_tx_done* y *rmt_config.* También había una función llamada *ws2812_rmt_adapter* que convertía manualmente los datos del buffer en la secuencia de pulsos que el LED necesita.

Con la nueva API, todo eso cambia bastante. En lugar de usar un canal como un simple identificador (rmt_channel_t), ahora se trabaja con *rmt_channel_handle_t,* que es un handle a un objeto del driver. Además, se incorpora un rmt_encoder_handle_t, que se encarga de hacer automáticamente la conversión de los datos a la señal RMT. Gracias a esto, ya no hace falta usar *rmt_item32_t* ni la función *ws2812_rmt_adapter*, por lo que toda esa parte se eliminó.

En la inicialización también hubo cambios importantes. Antes se usaban *rmt_config* y *rmt_driver_install*, mientras que ahora se crea el canal con *rmt_new_tx_channel,* se habilita con *rmt_enable* y se configura el encoder con *rmt_new_bytes_encoder.* Este encoder define cómo se representan los bits 0 y 1 en términos de niveles y duración, reemplazando completamente el cálculo manual de tiempos que se hacía antes con defines.

En la función ws2812_refresh, que es la que envía los datos al LED, se reemplazó *rmt_write_sample* por *rmt_transmit*, y la espera a que termine la transmisión ahora se hace con *rmt_tx_wait_all_done*. Esto hace que el envío del buffer sea más directo, ya que el encoder se encarga de traducir los datos automáticamente.

Por otro lado, funciones como ws2812_set_pixel prácticamente no cambiaron, ya que siguen escribiendo los valores de color en el buffer (en formato GRB). Lo mismo pasa con ws2812_clear, que simplemente pone el buffer en cero, y ws2812_del, donde solo se agregó la deshabilitación del canal RMT antes de liberar la memoria.

En cuanto a la implementación, se utilizan las funciones de las librerías delay y rgb_led en el main, para facilitar mucho más el funcionamiento de la secuencia del LED. Asimismo, la librería rgb_led utiliza el driver ya mencionado para implementar las funciones de manera más directa. Se logran distintos niveles, resultando más fácil o natural trabajar en el main utilizando las librerías intermediarias.

El control del LED WS2812C involucra dos aspectos diferentes. Por un lado, en el laboratorio se utiliza la función *esp_rom_delay_us()* para definir los tiempos durante los cuales el LED permanece en cada estado (rojo, verde, azul o apagado). Estos retardos son del orden de milisegundos y afectan el comportamiento visible del parpadeo, por lo que pueden implementarse por software sin inconvenientes, aunque se trate de una solución bloqueante.

Por otro lado, la comunicación con el LED se realiza utilizando un protocolo que requiere generar pulsos con tiempos muy precisos, del orden de los nanosegundos. Intentar generar esta señal directamente por software, controlando el GPIO con delays, no es confiable, ya que incluso funciones como *esp_rom_delay_us()* no tienen la resolución suficiente ni garantizan una temporización exacta.

Para resolver esto, se utiliza el periférico RMT del ESP32-S2, que permite generar automáticamente estas secuencias de pulsos con gran precisión mediante hardware. Entre sus ventajas se destacan la exactitud en los tiempos, la reducción de la carga del CPU y la independencia de la ejecución del programa principal, lo que asegura un funcionamiento estable del LED.

En resumen, los delays por software se utilizan para controlar el tiempo entre cambios de estado visibles del LED, mientras que el periférico RMT se encarga de generar la señal precisa sobre el GPIO necesaria para transmitir los datos según el protocolo del WS2812C. [6]

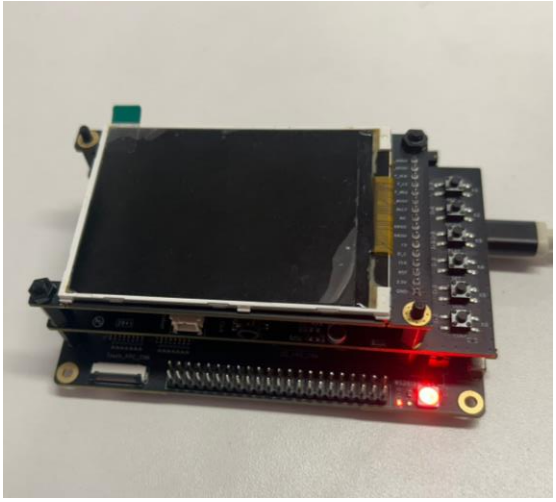

Figura 7: imagen de la placa ESP32-S2 funcionando.

Si se corre el programa con un delay de 500ms, al flashear se obtiene lo que aparece en la Figura 7. Una placa funcionando a partir de un programa que enciende en bucle la LED de color rojo, verde y azul. Se evaluó el monitor, y mostro correctamente en simultaneo el nombre del color que mostraba la placa.

Problemas encontrados:

Si se aumenta el delay entre cada color de 500ms a 10000ms, aunque la placa funcionaba correctamente (cambiando de color cada 10 segundos), al abrir el monitor saltaba un warning del watchdog. Indicaba que el programa quedo demasiado tiempo ocupando CPU dentro del main, sin ceder tiempo al sistema operativo. Entonces ESP-IDF detecta "la tarea IDLE no pudo correr" aparece el warning: *task watchdog got triggered.*

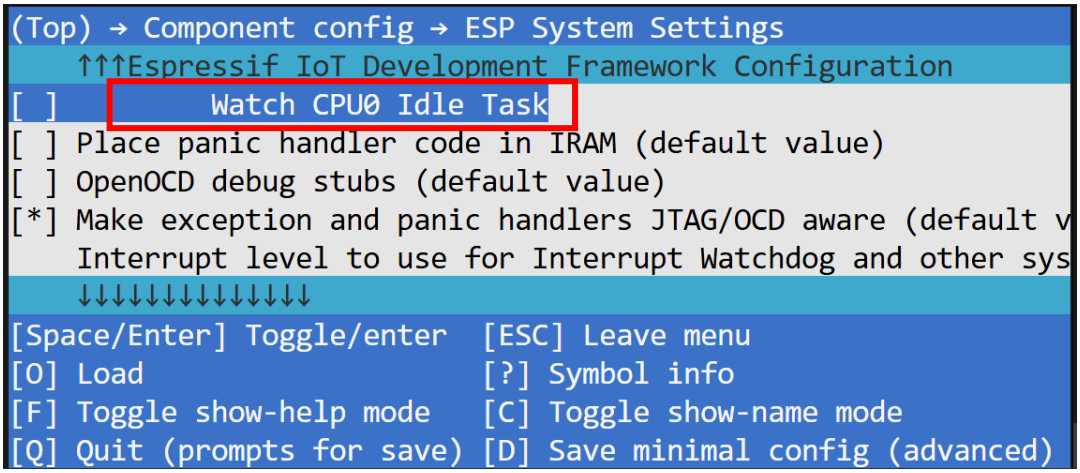

Figura 8: imagen de la configuración en la terminal de VSCode.

Se evaluó cambiar el timeout watchdog, pero al probarlo seguía saltando el mismo error. Se procedió a desactivar el watchdog de IDLE task en menuconfig como se muestra en la *Figura 8*. Esto quiere decir que deja de monitorear si la tarea interna de reposo del sistema recibe tiempo de CPU.

Conclusión

En esta segunda parte del laboratorio se trabajó con el control de un LED RGB WS2812C usando ESP-IDF. Primero se analizó cómo funciona su protocolo de comunicación, entendiendo que los datos se envían en serie y que el LED interpreta los bits según la duración de los pulsos, sin necesidad de un clock.

A partir de esto, se planteó un pseudocódigo para enviar bits, bytes y colores, lo que ayudó a entender mejor cómo se realiza la transmisión de datos hacia el LED. Esto sirvió como base para después implementar el control real desde el microcontrolador.

Luego se desarrollaron librerías propias (*rgb_led y delay*) organizadas como componentes dentro del proyecto, lo que permitió estructurar mejor el código y facilitar su uso desde el *main*. Gracias a esto, se pudo implementar el parpadeo del LED en distintos colores de forma más clara y ordenada.

Para el funcionamiento, se utilizó *esp_rom_delay_us()* para controlar los tiempos visibles del parpadeo, mientras que el periférico RMT se encargó de generar la señal precisa necesaria para el protocolo del WS2812C. Esto permitió ver la diferencia entre usar delays por software y usar hardware para señales que requieren mayor precisión. Además, se pudo visualizar el rol del watchdog en la detección de bloqueos en la ejecución, mencionado al final de esta segunda parte.

En conclusión, esta parte del laboratorio permitió entender conceptos importantes sobre comunicación con dispositivos externos, manejo de periféricos y organización del código en sistemas embebidos, destacando la importancia de utilizar correctamente los recursos de hardware para garantizar un funcionamiento confiable.

Bibliografía:

1. *Build System - ESP32 - — ESP-IDF Programming Guide latest documentation*. (s. f.). https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-guides/build-system.html

2. *Partition Tables - ESP32 - — ESP-IDF Programming Guide latest documentation*. (s. f.). https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-guides/partition-tables.html

3. *Project Configuration (sdkconfig) - ESP32 - — ESP-IDF Programming Guide latest documentation*. (s. f.). https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/kconfig-reference.html

4. *ESP32-S2-Kaluga-1 Kit - ESP32-S2 - — ESP-IDF Programming Guide v4.2-beta1 documentation*. (s. f.). https://docs.espressif.com/projects/esp-idf/en/v4.2-beta1/esp32s2/hw-reference/esp32s2/user-guide-esp32-s2-kaluga-1-kit.html

5. *Reference Documentation — esp-dev-kits documentation*. (s. f.). https://espressif-docs.readthedocs-hosted.com/projects/esp-dev-kits/en/latest/esp32s2/esp32-s2-kaluga-1/reference/reference-doc.html

6. www.alldatasheet.com. (s. f.). *WS2812C datasheet(3/8 Pages) WORLDSEMI*. Intelligent Control LED Integrated Light Source. https://www.alldatasheet.com/html-pdf/1134531/WORLDSEMI/WS2812C/1712/3/WS2812C.html

7. Letra de Laboratorio 1: Introducción a ESP-IDF y control básico del ESP32-S2